INDEX

Exp. No	Date	Title of the Experiments	Date of Submission	Page No.	Marks	Faculty In Charge Signature
0	25/May	Introduction		1		
1	25/May	a) Illustrate where clause AND or operations		3		
		(b) Execute insert, Query, update Delete and Projection commands		5		
2	28/May	(a) Develop a Query to select fields and ignore some fields.		8		
		(b) Develop a query to display 5 documents from the result		8		
3	11/June	(a) Execute Query selectors & comparisons and logical & list out the result		10		
		(b) Execute general and bitwise selectors & list out the results		11		
4	11/June	Create & demonstrate $elements match, $slice operations		14		
5	2/July	Execute aggregate $avg, $min, $max, $push, $addToSet, operators		16		

Experiment No.: D D M M Y Y Y Y

Faculty In Charge Signature

MongoDB

DataBase

A database is a structured collection of data that is organized and stored in a computer system. It allows for efficient storage, retrieval and manipulation of data. Databases are used in a wide range of applications, including websites, business applications, scientific research, and more.

There are various types of databases, including:

(i) Relational database: These are all the most common type of database, where data is organised into tables with rows and columns. Relational databases use structured Query language (SQL) for querying and managing data.

Examples include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

(ii) NoSQL Databases: NoSQL (Not only SQL) databases are designed to handle large volumes of unstructured or semi-structured data. They provide more flexibility and scalability compared to relational databases and are often used in big and real-time web applications. Examples including MongoDB, Cassandra, Redis and Couchbase.

(iii) Graph Databases: Graph databases are designed to represent and store data as nodes, edges, and properties, which are interconnected. They are used for applications where relationships between data points are important, such as social networks, recommendation systems, and network analysis. Examples include Neo4j, Amazon Neptune, and ArangoDB.

(iv) Document Databases: Document Databases store and retrieve data in a semi-structured format, typically using JSON or XML documents. They are suitable for content management systems, e-commerce platforms, and other applications where data varies

www.cityengineeringcollege.ac.in

~~true, size: 2, max:~~ ~~ok: 1~~) in structure. Examples include MongoDB, Couchbase, and RavenDB.

(v) Columnar Databases: Columnar Databases store data in columns rather than rows, which allows for efficient data retrieval and analysis, especially for analytical queries on large datasets. They are commonly used in data warehousing and business intelligence applications.

Introduction to MongoDB

- MongoDB is a document-oriented NoSQL database.
- Developed by MongoDB Inc.
- Initially released ~~from~~ in 2009
- Written in C++, C.

Use Cases:

- Content Management: MongoDB's flexible schema and scalability make it suitable for content management systems handling diverse types of content.
- Real-Time Analytics: MongoDB's high performance read and write operations make it suitable for real-time analytics and logging applicat.
- Catalogue and Product Data: MongoDB can handle complex product catalogues with varying attributes and structure efficiently.
- Mobile and Social Infrastructure: MongoDB is well-suited for mobile and social applications due to its ability to handle large volumes of unstructured data
- IOT Data Storage: MongoDB can efficiently store and analyze large volumes of data generated by IOT devices.

Experiment - 1

a Aim: Illustration of Where Clause, AND, OR operations in MongoDB.

Theory: • Create a new database.
• Use {database name
• To show all the databases show dbs
• Create collection
• Insert the required documents
• find()
To select the data from a collection in MongoDB, we can use the find() method. This method accepts a query object. If left empty, all documents will be returned
• findOne()
To select only one document, we can use the findOne() method
In MongoDB, the 'find' method is used to query the databases. The 'where' clause is implicitly included in the Query object
• AND operation
To perform an AND operation in MongoDB, we can specify multiple conditions in the query object. All conditions must be true for a document to be included in the result.
• OR operation
To perform an OR operation, you use '$or' operator and provide an array of conditions. At least one of the conditions must be true for a document to be included in the result

Experiment No.:

D D M M Y Y Y Y

Program:

a) db.collection.find({$and:[{age:{$gt:25}}, {gender:"male"}]})

(ii) db.collection.find({$or:[{age:{$gt:30}}, {gender:"Female"}]})

Output:

(i) [{
_id: ObjectID("66512K1d6e2uebbfc46b79a"),
Name: 'Joh',
age: 35,
gender: 'Male'
},
{
_id: ObjectID("66512258d6e2uebbfc46b79c"),
Name: 'Arhan',
age: 56
gender: 'Male',
}]

(ii) [{ _id: ObjectID("66512ic1d6e2uebbc46b79a"),
Name: 'John',
age: 35
gender: 'Male',
},
{ _id: ObjectID("66512220d6e2uebbfc46b79b")
Name: 'Kavya',
age: 53
gender: 'Female'
}]

Experiment No.:

D D M M Y Y Y Y

(b) Aim: Execute the commands of MongoDB and operations in MongoDB: Insert, Query, Update, Delete and Projection.

Theory:

- Insert: Use 'insertOne' and 'insertMany' to add documents.
- Query: Use 'find' to retrieve documents.
- Update Document: To update an existing document we can use the updateOne() or updateMany() methods. The first parameter is a query object to define which document or documents should be updated. The second parameter is an object defining the updated data.
- Delete: Use 'deleteOne' and 'deleteMany' to remove documents
- Projection: Both find methods accept a second parameter called projection. This parameter is an object that describes which fields to include in the results.

Program:

~~(a) To insert~~ db.createcollection("Studentdetails")

(i) To insert one document

db.studentdetails.insertOne({title: "Post title 1", body: "Body of post", category: "News", likes: 1, tags: ["news", "events"], date: Date()})

(ii) To insert multiple documents:

db.posts.insertMany([
{ title: "Post title 2", body: "Body of Post", category: "Event", likes: 2, tags: ["news", "events"], date: Date()},
{title: "Post Title 3", body: "Body of Post", category: "Technology", likes: 3, tags: ["news", "events"], date: Date()},
{title: "Post Title 4", body: "Body of post", category: "Event",

likes:4, tags: ["news","events"], date: Date()}])

(iii) Update Document

db.posts.find({title:"Post Title 1"})

db.posts.updateOne({title:"Post Title 1"}, {$set:{likes:2}})

db.posts.updateMany({}, {$inc:{likes:1}})

(iv) Delete One document

db.collectionName.deleteOne({key:value})

db.student.deleteOne({Anna:101})

(v) Delete Multiple Documents

db.collectionName.deleteOne({key:value})

db.student.deletMany({age:{$gt:30}})

(vi) Projection:

db.posts.find({}, {title:1, date:1})

Output,

{ok:1}

(i) { "acknowledged": true, "insertId": ObjectId("...")}

(ii) {"acknowledged": true, "insertIds": [ObjectId("..."), ObjectId("..."), ObjectId("...")]}

(iii) [{ "_id": ObjectId("..."), "title": "Post Title 1", "body": "Body of post.", "category": "News", "likes": 1, "tags": ["news","events"], "date": ISODate("2024-06-17")}]

(iv) {"acknowledged": true, "matchedCount": 1, "modifiedCount": 1}

{"acknowledged": true, "matchedCount": 4, "modifiedCount": 4}

(iv) {"acknowledged": true, "deletedCount": 1}

{"acknowledged": true, "deletedCount": 1}

Experiment No.:

D D M M Y Y Y Y

(v) {"acknowledged": true, "deletedCount": 2}

(vi) [
{"_id": ObjectId("...."), "title": "Post Title 1", "date": ISODate("2024-06-17")},
{"_id": ObjectId("...."), "title": "Post Title 2", "date": ISODate("2024-06-17")},
{"_id": ObjectId("...."), "title": "Post title 3", "date": ISODate("2024-06-17")},
{"_id": ObjectId("...."), "title": Post title 4", "date": ISODate("2024-06-17")
}]

Experiment - 2

(a) Develop a MongoDB query to select certain fields and ignore some fields of the documents from any collection

Theory:

To address the task of selecting specific fields and ignoring others in documents from any MongoDB collection, Develop a MongoDB query that:

(i) Select Certain fields: This means to explicitly specify which fields you want to be included in the query results.

(ii) Ignores some fields: This means to explicitly specify which fields you want to be excluded from the query results.

Program:

```
db.collection.find({}, {Name:1, age:1, _id:0})
```

Output:

```
[{
  "Name": "John", "age": 25},
  {Name: "Ishitha" age: 30},
  {Name: "Charlie", age: 28}
]
```

(b) Aim: Develop a MongoDB query to display the first 5 documents from the results obtained in a use of limit and find.

Theory:

To display the first 5 documents from the results obtained in the previous query, we use the 'limit' method along with

the 'find' method in MongoDB. The 'limit' method restricts the number of documents returned by the query.

~~outp~~ Program:

```
db.collection.find().limit(5)
```

Output:

```
[
{Name: "John", age: 25},
{Name: "Ishitha", age: 30},
{Name: "Charlie", age: 28},
{Name: "David", age: 35},
{Name: "Eve", age: 22}
]
```

Experiment - 3

(a) Aim:- Execute query selectors (comparison selectors, logical selectors) and list out the results on any collection.

Theory:

(i) Comparison Selectors: Comparison selectors are used to filter documents based on the values of fields.

Example: • $gt: Greater than, • $lt: Less than
• $gte: Greater than or equal to • $ne: Not equal to
• $lte: Less than or equal to • $eq: Equal to

(ii) Logical Selectors: Logical selectors allow combining multiple conditions.

Example: • $and: Logical AND, returns documents that match all the conditions.
• $or: Logical OR, returns documents that match any of the conditions.
• $not: Inverts the effect of a Query expression
• $nor: Logical NOR, returns documents that fail to match both clauses.

Program

(i) db.collection.find({age: {$gt: 30}})

(ii) db.collection.find({$and: [{age: {$gt: 25}}, {gender: "male"}]})

Output

(i) [
{Name: "John", age: 35, gender: "male"},
{Name: "Charlie", age: 32, gender: "male"}
]

www.cityengineeringcollege.ac.in

(ii) [
{Name: "John", age: 35, "gender": "male"},
{Name: "Charlie", age: 28, gender: "male"},
{Name: "David", age: 32, gender: "male"}
]

(b) Execute query selectors (Geospatial selectors, Bitwise selectors and list out the results on any collection.

b1) Geospatial selectors

(i) Set up MongoDB and your Collection: Ensure you have MongoDB installed and running. Create a database and collection where you want to store your geospatial data.

(ii) Define Geospatial Data: MongoDB supports geospatial data in two formats:

- GeoJSON Objects: This is a standard format for representing geospatial data.
- Legacy Coordinate Pairs: These are simple coordinate pairs (longitude, latitude).

(iii) Create a Geospatial Index: Before inserting geospatial index on the collection

(iv) Insert Geospatial Data: Insert your geospatial data into the collection.

Program

(i) Create a Database and Collection:

```
use geospatialDB
db.createCollection("locations")
```

(ii) Create a Geospatial Index:

```
db.locations.createIndex({location: "2dsphere"})
```

(iii) Insert Geospatial ~~Index~~ Data:

db.locations.insertMany([{ name: "Location A", location: {type: "Point", coordinates: [-73.97, 40.77]}}
{name: "Location B", location: {type: "Point", coordinates: [-73.88, 40.78]}}])

(iv) Find Nearest Locations:

db.locations.find({location: {$near: {$geometry: {type: "Point", coordinates: [-73.95, 40.75]}, $maxDistance: 5000}}})

Output,

[
{ _id: ObjectId("..."), Name: "Location A", location: {"type": "Point", "coordinates": [-73.97, 40.77]}},
{ _id: ObjectId("..."), Name: "Location B", location: {"type": "Point", "coordinates": [-73.88, 40.78]}}]

b2) Bitwise selectors:

Theory:

- $bitsAllClear: Matches numeric or binary values in which a set of bit positions all have a value of 0
- $bitsAllSet: Matches numeric or binary values in which a set of bit positions all have a value of 1
- $bitsAnyClear: Matches numeric or binary values in which any bit from a set of bit positions has a value of 0.
- $bitAnySet: Matches numeric or binary values in which any bit from a set of bit positions has a value of 1

In MongoDB, bitwise operators can be used within query expressions to perform bitwise operations on integer values.

- Bitwise AND ($bitAllSet): This operator selects documents where a

a given field's bit positions all have bits set (i.e., are 1)

- Bitwise OR ($bitsAnySet): This operator selects documents where any of the given bit positions are set.
- Bitwise AND with Complement ($bitsAllClear): This operator selects documents where the specified bit positions are clear (i.e., are 0).
- Bitwise OR with Complement ($bitsAnyClear): This operator selects documents where any of the specified bit positions are clear.

Program,

```
db.collection.insertMany([
  {"_id":1, "field":5}, {"_id":2, "field":9}, {"_id":3, "field":12}])
```

(i) db.collection.find({field:{$bitsAllSet:[0,2]}})
(ii) db.collection.find({field:{$bitsAnySet:[1]}})
(iii) db.collection.find({field:{$bitsAllClear:[0,2]}})
(iv) db.collection.find({field:{$bitsAnyClear:[0,2]}})

Output,

(i) [{"id":1, "field":5}]
(ii) [{"_id":3, "field":12}]
(iii) []
(iv) [{"_id":2, "field":9},
{"_id":3, "field":12}
]

Experiment - 4

Aim: Create and demonstrate how projection operators ($, $elemMatch and $slice) would be used in the MongoDB.

Theory: Projection Operators

- $: The positional $ operator limits the contents of an <array> to return the first element that matches the query condition on the array.
- $elemMatch: The $elemMatch operator limits the contents of an <array> field from the query results to contain only the first element matching the $elemMatch condition.
- $slice: The $slice projection specifies the number of elements in an array to return in the query result.

Program,

```
db.marklist.insertMany([
{"_id": 1, "semester": 1, "grades": [70, 87, 90]},
{"_id": 2, "semester": 1, "grades": [90, 88, 92]},
{"_id": 3, "semester": 1, "grades": [85, 100, 90]},
{"_id": 4, "semester": 2, "grades": [79, 85, 80]},
{"_id: 5, "semester: 2", "grades": [88, 88, 92]},
{"_id: 6, "semester: 2" "grades": [95, 90, 96]}])

(i) db.marklist.find({semester: 1, grades: {$gte: 85}}, {"grades$": 1})
(ii) db.marklist.find({"grades": {"$elemMatch": {"$gte": 95}}})
(iii) db.marklist.find({}, {"grades": {"$slice": 1}})
```

Output,

(i) [
{"_id": 1, "grades": [87]},
{"_id": 2, "grades": [90]},
{"_id": 3, "grades": [85]}]

(ii) [{"_id": 6, "Semester": 2, "grades": [95, 90, 96]}]

(iii) [
{"_id": 1, "Semester": 1, "grades": [70]},
{"_id": 2, "semester": 1, "grades": [90]},
{"_id": 3, "semester": 1, "grades": [85]},
{"_id": 4, "Semester": 2, "grades": [79]},
{"_id": 5, "semester": 2, "grades": [88]},
{"_id": 6, "semester": 2, "grades": [95]}
]

Experiment-5

Aim: Execute aggregation operation ($avg, $min, $max, $push $addToSet etc).

Theory:

- $avg: Returns the average value of the numeric values. It ignores non-numeric values.
- $max: Returns the maximum value of. $max compares both value and type using the specified BSON comparison order for values of different types.
- $min: Returns the minimum value, $min compares both value and type, using the specified BSON comparison order for values of different types.
- $push: Returns array of all unique values that result from applying an expression to documents.
- $addToSet: $addToSet returns an array of all unique values that results from applying an expression to each document is group

Applications:

- Real-Time analytics
- E-commerce

Program:

```
db.sales.insertMany([{"_id":1, "item":"abc", "price":10,
"quantity":2, "date": ISODate("2014-01-01T05:00:00Z")},
{"_id":2, "item": "jkl", "price":20, "quantity":1, "date":
ISODate("2014-02-03T09:00:00Z")},
{"_id":3, "item": "xyz", "price":5, "quantity":5, "date":
ISODate("2014-02-03T09:05:00Z")},
```

Experiment No.:

D D M M Y Y Y Y

{"_id": 4, "item": "abc", "price": 10, "quantity": 10, "date": ISODate("2014-02-15T08:00:00Z")},
{"_id": 5, "item": "xyz", "price": 5, "quantity": 10, "date": ISODate("2014-02-15T09:05:00Z")},
{"_id": 6, "item": "xyz", "price": 5, "quantity": 5, "date": ISODate("2014-02-15T12:05:10Z")},
{"_id": 6, "item": "xyz", "price": 5, "quantity": 10, "date": ISODate("2014-02-15T14:12:12Z")}])

(1) $avg:
db.sales.aggregate([{$group: {_id: null, averageQuantity: {$avg: "$quantity"}, count: {$sum: 1}}}])

(2) $min:
db.sales.aggregate([{$group: {_id: null, minqty: {$min: "$quantity"}}}])

(3) $max:
db.sales.aggregate([{$group: {_id: null, maxqty: {$max: "$quantity"}}}])

(4) $push:
db.sales.aggregate([{$group: {_id: {day: {$dayOfYear: "$date"}, year: {$year: "$date"}}, itemsSold: {$push: {item: "$item", quantity: "$quantity"}}}}])

Output,

(5) $addToset
db.sales.aggregate([{$group: {_id: {day: {$dayOfYear: "$date"}, year: {$year: "$date"}}, itemsSold: {$addToset: "$date"}}

www.cityengineeringcollege.ac.in

"$item" }}}])

Output,
{ acknowledged : true,
insertIds : { '0' : 1 , '1' : 2 , '2' : 3 , '3' : 4 , '4' : 5 , '5' : 6 , '6' : 7 }
}

(1) [{ _id : null , averageQuantity : 6.142857142857143, count : 7 }]

(2) [{ _id : null , minqty : 1 }]

(3) [{ _id : null , maxqty : 10 }]

(4) [{ _id : { day : 1 , year : 2014 },
itemsSold : [{ item : 'abc' , quantity : 2 }]},
{ _id : { day : 34 , year : 2014 }, itemsSold : [{ item : 'jkl' ,
quantity : 1 }, { item : 'xyz' , quantity : 5 }]},
{ _id : { day : 46 , year : 2014 },
itemsSold : [
{ item : 'abc' , quantity : 10 }
{ item : 'xyz' , quantity : 10 }
{ item : 'xyz' , quantity : 5 }
{ item : 'xyz' , quantity : 10 }]}]

(5) { _id : { day : 1 , year : 2014 }, itemsSold : ['abc']},
{ _id : { day : 34 , year : 2014 }, itemsSold : ['xyz , jkl']},
{ _id : { day : 46 , year : 2014 }, itemsSold : ['xyz' , 'abc']}]

www.cityengineeringcollege.ac.in